

ElevateX Project – Task Breakdown for 5 Team Members

Project Overview

ElevateX is an Employee Engagement Platform with the following core modules:

1. Authentication & User Management
2. Events Management
3. Submissions & Approvals
4. Gamification (Points/Wallet)
5. Leaderboards & Analytics

Each team member will own **one complete module** (both frontend and backend), ensuring seamless end-to-end ownership.

Team Allocation

Member 1:→ Auth & User Management

Member 2:→ Events Management

Member 3:→ Submissions & Approvals

Member 4:→ Gamification (Wallet/Points)

Member 5:→ Leaderboards & Analytics

Detailed Task Breakdown

MODULE 1: AUTHENTICATION & USER MANAGEMENT

Overview

Complete user lifecycle management including registration, login, profile management, and role-based access control.

Backend Tasks

APIs to Build

- **POST /api/auth/register** - User registration with validation
- **POST /api/auth/login** - User login, JWT token generation
- **POST /api/auth/logout** - Token invalidation
- **GET /api/auth/me** - Get current user details
- **GET /api/users** - List all users (Admin only)
- **GET /api/users/{user_id}** - Get user profile
- **PUT /api/users/{user_id}** - Update user profile
- **PUT /api/users/{user_id}/role** - Update user role (Admin only)
- **DELETE /api/users/{user_id}** - Delete user (Admin only)
- **POST /api/auth/refresh-token** - Refresh JWT token

Services to Implement

- `UserService` - CRUD operations, role management
- Update `auth.py` - Verify token, get current user
- Role-based middleware - Admin, Manager, Employee checks
- Password reset flow (email integration)

Models to Create

- User model with fields: `_id`, `name`, `email`, `password_hash`, `role`, `department`, `created_at`, `updated_at`
- `UserCreate`, `UserUpdate`, `UserLogin` schemas
- Token model with `access_token`, `token_type`, `user`

Database

- MongoDB `users` collection with indexes on `email`, `role`
- Password hashing with `bcrypt`
- JWT secret key in `.env`

Testing

- Unit tests for authentication functions
- Integration tests for login/registration flow
- Test role-based access control

Frontend Tasks

Pages to Build

- **Login.jsx** - Email/password login form with error handling
- **Register.jsx** - Registration form with validation
- **Profile.jsx** - User profile view and edit
- **Users.jsx** (Admin) - User list, role assignment, actions

Components to Build

- LoginForm component with form validation
- RegistrationForm component
- ProfileCard component
- UsersList component with action buttons
- RoleSelector component (dropdown/select)

Features to Implement

- Form validation (email, password strength)
- Password show/hide toggle
- Error messages display
- Success notifications
- Redirect to dashboard after login
- AuthContext state management
- Protected routes (guards)
- Role-based UI rendering

Styling

- Login.css - Professional login layout
- Register.css - Clean registration form
- Profile.css - Profile card styling
- Admin user management styling

API Integration

- axios calls to `/api/auth/register`
- axios calls to `/api/auth/login`

- axios calls to `/api/users/{id}`
- Token storage in localStorage
- Automatic token refresh

Dependencies

- ☒ No external module dependencies
- Standalone module
- Must be completed first (other modules depend on auth)

Deliverables

- ☒ Complete authentication flow (register, login, logout)
- ☒ User profile management
- ☒ Role-based access control implemented
- ☒ JWT token management
- ☒ Password hashing & security
- ☒ Protected routes on frontend
- ☒ User UI pages (Login, Register, Profile, Admin Users)
- ☒ Full test coverage

Acceptance Criteria

- ☐ User can register with email validation
 - ☐ User can login and receive JWT token
 - ☐ User can logout and token is invalidated
 - ☐ User can view and edit their profile
 - ☐ Admin can manage all users and roles
 - ☐ JWT tokens expire after 24 hours
 - ☐ All forms have proper validation
 - ☐ Error messages are clear and helpful
-



MODULE 2: EVENTS MANAGEMENT

Overview

Complete event lifecycle from creation to completion, including event CRUD, filtering, and event categorization.

Backend Tasks

APIs to Build

- **GET /api/events** - List all events (with filters: department, difficulty, type, status)
- **GET /api/events/{event_id}** - Get event details
- **POST /api/events** - Create event (Manager/Admin only)
- **PUT /api/events/{event_id}** - Update event (Manager/Admin)
- **DELETE /api/events/{event_id}** - Delete event (Admin only)
- **GET /api/events/{event_id}/participants** - Get event participants
- **PUT /api/events/{event_id}/status** - Change event status (draft→active→closed→archived)
- **GET /api/events/search** - Search events by keyword, tags

Services to Implement

- `EventService` - CRUD operations, filtering, search
- Event validation (deadline > today, positive reward points)
- Participant management (add/remove)
- Event status workflow
- Tag management

Models to Create

- Event model: `_id`, `title`, `description`, `type`, `department`, `difficulty`, `deadline`, `reward_points`, `max_participants`, `current_participants`, `status`, `tags`, `created_by`, `created_at`, `updated_at`
- EventCreate, EventUpdate schemas
- Event status enum: draft, active, closed, archived

Database

- MongoDB `events` collection
- Indexes on: `department`, `status`, `deadline`, `created_by`
- Tags array with text search capability

Business Logic

- Validate event deadline (must be future date)
- Auto-close events past deadline
- Check participant limits
- Calculate remaining slots
- Tag-based categorization

Testing

- CRUD operation tests
- Filter and search tests
- Status transition tests
- Permission tests (only manager/admin can create)

Frontend Tasks

Pages to Build

- **Events.jsx** - Event listing with filters and search
- **EventDetail.jsx** - Single event details and join button
- **CreateEvent.jsx** (Admin/Manager) - Event creation form
- **EditEvent.jsx** (Admin/Manager) - Event edit form

Components to Build

- EventCard component - Event preview card
- EventGrid component - Grid/list layout
- EventFilter component - Filter by department, difficulty, type
- EventSearch component - Search bar
- EventForm component - Reusable form (create/edit)
- EventTags component - Display tags
- JoinEventModal component - Confirm participation
- ParticipantsList component - View event participants

Features to Implement

- Display all events with pagination
- Filter by department, difficulty, type, status
- Search events by title/description
- Event detail modal/page
- Join/leave event functionality
- Create/edit events (managers only)
- Event status badges (Active, Closed, Archived)
- Deadline countdown
- Reward points display
- Tag display and filtering
- Participant count display

Styling

- Events.css - Grid/list layout
- EventCard.css - Card styling
- EventDetail.css - Detail page styling
- Admin event forms styling

API Integration

- GET `/api/events` with filters
- GET `/api/events/{id}`
- POST `/api/events` (create)
- PUT `/api/events/{id}` (edit)
- DELETE `/api/events/{id}` (delete)
- Event filtering state management

Dependencies

-  **Depends on:** Auth & User Management (Module 1)
 - Must have authenticated users and roles
 - Manager/Admin role checks required

Deliverables

-  Complete event management system
-  Event CRUD operations
-  Advanced filtering and search
-  Event status workflow
-  Event listing UI with pagination
-  Event creation/editing forms
-  Role-based event management
-  Tag system for categorization

Acceptance Criteria

- ☐ Users can view all available events
- ☐ Events can be filtered by department, difficulty, type
- ☐ Events can be searched by title/description/tags
- ☐ Managers/Admins can create events with all fields
- ☐ Event deadline must be in the future
- ☐ Event status changes based on deadline
- ☐ Participants are limited to max_participants
- ☐ Event details show all relevant information

- ☐ Only creators/admins can edit events
-



MODULE 3: SUBMISSIONS & APPROVALS

Overview

Submission management system where employees submit work for events and managers/admins approve or reject.

Backend Tasks

APIs to Build

- **POST /api/submissions** - Create submission (upload files)
- **GET /api/submissions** - List submissions (with filters: status, event, user)
- **GET /api/submissions/{submission_id}** - Get submission details
- **PUT /api/submissions/{submission_id}** - Update submission (employee only, before review)
- **DELETE /api/submissions/{submission_id}** - Delete submission (employee only, before review)
- **PUT /api/submissions/{submission_id}/approve** - Approve submission (Manager/Admin)
- **PUT /api/submissions/{submission_id}/reject** - Reject submission with reason (Manager/Admin)
- **GET /api/users/{user_id}/submissions** - Get user's submissions
- **GET /api/events/{event_id}/submissions** - Get event's submissions
- **POST /api/submissions/{submission_id}/comments** - Add review comment

Services to Implement

- `SubmissionService` - CRUD, status management, approval workflow
- File upload handling and storage
- Submission validation (attached files, event participation)
- Approval/rejection logic with notifications
- Submission history tracking

Models to Create

- Submission model: `_id`, `event_id`, `user_id`, `description`, `file_urls`, `status`, `submitted_at`, `reviewed_at`, `reviewer_id`, `review_comments`, `rejection_reason`, `points_awarded`
- SubmissionCreate, SubmissionUpdate schemas
- Submission status enum: pending, approved, rejected, revision_requested

Database

- MongoDB `submissions` collection
- Indexes on: `event_id`, `user_id`, `status`, `submitted_at`
- File storage in `uploads` directory with references

File Management

- Handle file uploads (CSV, PDF, images)
- Store files in `/backend/uploads/` with unique naming
- Generate file URLs for download
- File size validation (max 50MB)

Business Logic

- Only enrolled users can submit
- One submission per user per event
- Can only edit before review starts
- Approval awards points to user
- Rejection requires reason comment
- Track submission history

Testing

- File upload tests
- Submission CRUD tests
- Approval/rejection workflow tests
- Permission tests

Frontend Tasks

Pages to Build

- **Submit.jsx** - Submission form with file upload
- **Submissions.jsx** (User) - User's submissions list
- **AdminApprovalsList.jsx** (Admin/Manager) - Submissions to review

Components to Build

- FileUpload component - Drag-drop file upload
- SubmissionForm component - Event selection + file upload
- SubmissionCard component - Submission preview

- SubmissionsList component - Table/list view
- ApprovalPanel component - Review form (approve/reject)
- SubmissionDetail component - Full submission view
- ReviewComments component - Comments thread
- StatusBadge component - Visual status indicator

Features to Implement

- File upload with drag-and-drop
- File size/type validation (frontend)
- Submit work for events
- View submission history
- Track submission status (pending, approved, rejected)
- Download submitted files
- Manager review interface
- Approve/reject submissions with comments
- Revision request workflow
- Notification on approval/rejection
- Submission timeline/history view

Styling

- Submit.css - Form styling
- Submissions.css - List view
- ApprovalsList.css - Review interface styling
- FileUpload.css - Drag-drop visual feedback

API Integration

- POST `/api/submissions` with file upload
- GET `/api/submissions` with filters
- GET `/api/users/{id}/submissions`
- GET `/api/events/{id}/submissions`
- PUT `/api/submissions/{id}/approve`
- PUT `/api/submissions/{id}/reject`
- File upload handling with FormData

Dependencies

-  **Depends on:**
 - Module 1 (Auth) - User authentication
 - Module 2 (Events) - Event selection, user enrollment

Deliverables

- ☒ Complete submission workflow
- ☒ File upload and storage system
- ☒ Approval/rejection management
- ☒ Submission tracking UI
- ☒ Manager review interface
- ☒ Comment/feedback system
- ☒ Status tracking

Acceptance Criteria

- ☐ Users can submit files for enrolled events
 - ☐ File upload supports multiple file types
 - ☐ File size is validated (frontend + backend)
 - ☐ Managers can review pending submissions
 - ☐ Managers can approve with points award
 - ☐ Managers can reject with reason
 - ☐ Users see submission status updates
 - ☐ Only one submission per event per user
 - ☐ Files are securely stored and accessible
 - ☐ Submission history is maintained
-

☒ MODULE 4: GAMIFICATION (POINTS & WALLET)

Overview

Points and rewards system including wallet management, point transactions, and redemption.

Backend Tasks

APIs to Build

- **GET /api/wallet** - Get user's current points balance
- **GET /api/wallet/history** - Get points transaction history
- **POST /api/wallet/redeem** - Redeem points for rewards
- **GET /api/wallet/leaderboard** - Get top users by points (basic)
- **GET /api/rewards** - List available rewards to redeem
- **POST /api/rewards** - Create reward (Admin only)

- **PUT /api/rewards/{reward_id}** - Update reward (Admin)
- **DELETE /api/rewards/{reward_id}** - Delete reward (Admin)
- **GET /api/users/{user_id}/points** - Get user's point balance

Services to Implement

- `WalletService` - Point balance management, transactions
- Point calculation and award logic
- Redemption processing and validation
- Transaction history tracking
- Reward management

Models to Create

- Wallet model: `_id`, `user_id`, `balance`, `total_earned`, `total_spent`, `last_updated`
- Transaction model: `_id`, `user_id`, `type` (award/redeem), `amount`, `source` (submission_approval/reward_redemption), `reference_id`, `created_at`
- Reward model: `_id`, `name`, `description`, `points_required`, `stock`, `is_active`, `created_at`
- WalletTransaction schema for API

Database

- MongoDB `wallets` collection with `user_id` index
- MongoDB `transactions` collection for transaction history
- MongoDB `rewards` collection for reward catalog
- Proper indexing for performance

Business Logic

- Award points on submission approval (from submission `reward_points`)
- Deduct points on redemption
- Prevent negative balance
- Validate redemption (sufficient points, reward exists, stock available)
- Track transaction history
- Generate point summaries
- Integration with submissions (award points on approval)

Testing

- Point calculation tests

- Redemption validation tests
- Transaction history tests
- Balance update tests

Frontend Tasks

Pages to Build

- **Wallet.jsx** - Wallet dashboard with balance and rewards store
- **RewardsStore.jsx** - Browse and redeem rewards
- **PointsHistory.jsx** (User) - Transaction history view

Components to Build

- WalletCard component - Display current balance
- PointsBadge component - Show points in nav
- RewardCard component - Reward preview card
- RewardsGrid component - Grid of available rewards
- RedeemModal component - Confirm redemption
- TransactionsList component - History table
- PointsBreakdown component - Earned vs spent breakdown
- LeaderboardPreview component - Top earners snippet

Features to Implement

- Display user's current point balance
- Show points earned breakdown
- Browse available rewards
- Redeem points for rewards with confirmation
- View transaction history with details
- Filter transactions (earned, spent)
- Reward stock availability
- Instant balance update after redemption
- Points goal/target progress
- Mini leaderboard preview (top earners)
- Point notifications

Styling

- Wallet.css - Dashboard styling
- RewardsStore.css - Grid and card styling
- TransactionsList.css - Table/list styling

- PointsBadge styling in navbar

API Integration

- GET `/api/wallet`
- GET `/api/wallet/history`
- GET `/api/rewards`
- POST `/api/wallet/redeem`
- Real-time balance updates
- Transaction filtering

Dependencies

-  **Depends on:**
 - Module 1(Auth) - User authentication
 - Module 3(Submissions) - Awards points on approval

Deliverables

-  Complete points/wallet system
-  Reward catalog and redemption
-  Transaction history tracking
-  Balance management
-  Wallet UI dashboard
-  Rewards store interface
-  Transaction history view

Acceptance Criteria

- ☐ Users can view their point balance
 - ☐ Points are awarded on submission approval
 - ☐ Users can redeem points for rewards
 - ☐ Redemption updates balance instantly
 - ☐ Sufficient points validation before redemption
 - ☐ Reward stock is managed
 - ☐ Transaction history is complete and accurate
 - ☐ Only admin can manage reward catalog
 - ☐ Negative balance is prevented
 - ☐ Points calculation is correct
-



MODULE 5: LEADERBOARDS & ANALYTICS

Overview

Real-time leaderboards, engagement metrics, and analytics dashboards.

Backend Tasks

APIs to Build

- **GET /api/leaderboard** - Global leaderboard (all users, top 100)
- **GET /api/leaderboard/department** - Department-specific leaderboard
- **GET /api/leaderboard/{user_id}/rank** - Get user's rank and context
- **GET /api/analytics/dashboard** - Main dashboard metrics
- **GET /api/analytics/engagement** - Engagement analytics (trends, participation)
- **GET /api/analytics/events** - Event-wise analytics
- **GET /api/analytics/user/{user_id}** - Individual user analytics
- **GET /api/analytics/departments** - Department-wise analytics
- **GET /api/analytics/reports** - Generate and list reports
- **POST /api/analytics/reports** - Generate new report

Services to Implement

- `LeaderboardService` - Ranking calculations, real-time updates
- `AnalyticsService` - Metrics aggregation, report generation
- Data aggregation from submissions, wallets, events
- Caching for performance (Redis optional)
- Report generation logic

Models to Create

- LeaderboardEntry: `user_id`, `rank`, `name`, `points`, `events_participated`, `submissions_approved`, `department`
- DashboardMetrics: `total_users`, `total_events`, `total_submissions`, `total_points_awarded`, `avg_participation_rate`
- Analytics data structure with metrics
- Report model for storing generated reports

Database

- MongoDB `leaderboard_entries` collection (denormalized for speed)

- Aggregation pipelines for calculations
- Proper indexing on `points`, `rank`, `department`
- Optional Redis caching for leaderboard

Business Logic

- Rank users by points (ties handled by `submission_count`, then by earliest completion)
- Department-wise rankings
- Engagement metrics calculation
- Participation rate computation
- Trend analysis (week-over-week, month-over-month)
- Auto-generate monthly reports (via scheduled job - coordinate with Module 1)
- Real-time leaderboard updates

Calculations

- User rank = ordered by points DESC, then submissions DESC
- Participation rate = $\text{submissions_approved} / \text{available_events}$
- Engagement score = $(\text{points} * 0.4) + (\text{submissions} * 0.3) + (\text{participation} * 0.3)$
- Department average = avg points in department

Testing

- Ranking algorithm tests
- Analytics calculation tests
- Report generation tests
- Performance tests for large datasets

Frontend Tasks

Pages to Build

- **Leaderboard.jsx** - Global leaderboard with filters
- **Analytics.jsx** - Main analytics dashboard
- **Dashboard.jsx** - Home dashboard with key metrics
- **Progress.jsx** (User) - Personal progress and analytics

Components to Build

- `LeaderboardTable` component - Ranked user list
- `RankCard` component - User's rank card
- `DepartmentFilter` component - Filter by department
- `MetricsCard` component - Display KPI cards

- ChartComponent (using Chart.js) - Various charts
 - Point distribution chart
 - Participation trend chart
 - Department comparison chart
 - Events completion rate chart
- ProgressBar component - User's engagement progress
- AchievementBadges component - Badges/achievements (optional)
- DashboardSummary component - Key metrics summary
- PersonalAnalytics component - User's personal stats

Features to Implement

- Global leaderboard with top 100 users
- Department-specific leaderboard
- User rank and position display
- Search/filter leaderboard
- Pagination for large lists
- Main dashboard with KPIs:
 - Total users
 - Total events created
 - Total submissions
 - Average participation rate
 - Total points distributed
- Engagement trends (line chart)
- Top performing users chart
- Department comparison chart
- Event completion rates
- User personal analytics:
 - Points earned over time
 - Events participated
 - Submissions status breakdown
 - Ranking progress
- Monthly report viewing/download
- Real-time updates for leaderboard
- Achievement badges (optional)

Styling

- Leaderboard.css - Table and filter styling
- Analytics.css - Dashboard grid styling
- Dashboard.css - Cards and charts layout
- Progress.css - Personal stats styling

- Chart styling for better visualization

Chart Libraries

- Chart.js with react-chartjs-2 (already in dependencies)
- Line charts for trends
- Bar charts for comparisons
- Pie charts for distributions

API Integration

- GET `/api/leaderboard` with pagination
- GET `/api/leaderboard/department`
- GET `/api/leaderboard/{id}/rank`
- GET `/api/analytics/dashboard`
- GET `/api/analytics/engagement`
- GET `/api/analytics/user/{id}`
- Periodically refresh leaderboard data

Dependencies

-  **Depends on:**
 - Module 1(Auth) - User authentication
 - Module 2 (Events) - Event data
 - Module 3 (Submissions) - Submission data
 - Module 4 (Wallet) - Points data

Deliverables

-  Complete leaderboard system (global & departmental)
-  Comprehensive analytics dashboard
-  User ranking and analytics
-  Engagement metrics and trends
-  Report generation system
-  Interactive charts and visualizations
-  Personal progress tracking

Acceptance Criteria

- ☐ Leaderboard displays users ranked by points
- ☐ Department-specific leaderboard works correctly

- ☐ User can see their rank and position
 - ☐ Dashboard shows all key metrics accurately
 - ☐ Charts display data correctly
 - ☐ Engagement trends are visible
 - ☐ Monthly reports are generated automatically
 - ☐ Analytics update in real-time
 - ☐ Pagination works for large datasets
 - ☐ Performance is acceptable (sub-2s load times)
-



Implementation Timeline

Phase 1: Foundation (Week 1)

- **Owner:** Rounak (Module 1)
- **Deliverable:** Complete Auth & User Management
- *Other members:* Setup dev environment, database, project structure

Phase 2: Core Features (Weeks 2-3)

- **Owner:** Arjun (Module 2)
- **Owner:** Priya (Module 3)
- Build Events + Submissions (can be done in parallel)

Phase 3: Gamification (Week 4)

- **Owner:** Vikram (Module 4)
- Integrate with completed modules

Phase 4: Analytics (Week 5)

- **Owner:** Neha (Module 5)
- Aggregate data from all modules

Phase 5: Testing & Integration (Week 6)

- All owners test their modules
- End-to-end testing
- Performance optimization

Phase 6: Deployment (Week 7)

- Deploy to staging
- Final testing
- Production deployment

Integration Points

Module	Depends On	Provides Interface To
Module 1(Auth)	None	All modules
Module 2 (Events)	Module 1	Module 3, 5
Module 3 (Submissions)	Module 1, 2	Module 4, 5
Module 4 (Wallet)	Module 1, 3	Module 5
Module 5 (Analytics)	Module 1, 2, 3, 4	All modules

Common Setup (All Members)

Development Environment

```
# Backend setup (runs on all machines)
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# Configure .env
MONGO_URI=mongodb://localhost:27017
DB_NAME=elevatex
SECRET_KEY=your-secret-key
ALGORITHM=HS256

# Run server
uvicorn main:app --reload

# Frontend setup
cd frontend
```

```
npm install
npm run dev # Runs on http://localhost:5173
```

Database

- MongoDB running locally (all members connect to same instance)
- Database name: `elevatex`
- Collections auto-created on first write

API Documentation

- FastAPI Swagger docs: <http://localhost:8000/docs>
- Share API contract before implementation

Git Workflow

```
# Each member works on their module branch
git checkout -b feature/module-{number}

# Regular commits
git add .
git commit -m "Module {n}: Specific task"

# Push to GitHub
git push origin feature/module-{number}

# Create PR when ready
# Others review before merge to main
```



Checklist Per Module

For Each Module Owner:

- ☐ Create feature branch: `feature/module-{number}`
- ☐ Setup initial file structure
- ☐ Create models and schemas
- ☐ Build backend APIs
- ☐ Write backend tests

- ☐ Build frontend components
- ☐ Implement frontend pages
- ☐ Implement API integration
- ☐ Test end-to-end flow
- ☐ Create PR with documentation
- ☐ Get code review approval
- ☐ Merge to main

Quality Standards

- ☐ Code follows project conventions
 - ☐ All functions have docstrings
 - ☐ Error handling implemented
 - ☐ Form validation complete
 - ☐ API endpoints documented
 - ☐ Tests pass (>80% coverage)
 - ☐ No console errors/warnings
 - ☐ Responsive design (mobile-friendly)
 - ☐ Performance acceptable
 - ☐ Security best practices followed
-



Communication Plan

Daily Standup (15 min)

- What did you complete yesterday?
- What are you working on today?
- Any blockers?

Weekly Integration Meeting

- Check integration points
- Resolve conflicts
- Discuss next week's goals

PR Review Process

- Code review by at least 1 team member
- Automated tests pass

- No merge without approval
-

Success Metrics

-  All modules completed on schedule
 -  All tests passing
 -  Code coverage >80%
 -  Zero critical bugs
 -  All features working end-to-end
 -  Performance benchmarks met
 -  Documentation complete
 -  Ready for production deployment
-

Resources

Documentation

- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://react.dev/>
- MongoDB: <https://docs.mongodb.com/>
- Axios: <https://axios-http.com/>
- Chart.js: <https://www.chartjs.org/>

Tools

- Postman/Insomnia - API testing
- VS Code - Code editor
- MongoDB Compass - Database GUI
- Git/GitHub - Version control

Testing

- pytest - Python testing
 - Jest - JavaScript testing (optional)
 - Postman - API integration testing
-

Project Start Date: April 3, 2026

Expected Completion: May 15, 2026 (6 weeks)

Repository: <https://github.com/Zenith-11/ElevateX>